



合肥师范学院
HEFEI NORMAL UNIVERSITY

实验报告

课程名称: web 程序设计

学院: 计算机与人工智能学院

专业: 计算机科学与技术

学号: 2410211124

姓名: 朱品赞

指导教师: 张由明

2025 年 12 月 26 日

实验三 JDBC 技术实验

一、实验目的

- 1. 掌握 JDBC 的基本原理和使用方法
- 2. 学会使用数据库连接池（Druid）实现连接管理
- 3. 理解 DAO 设计模式，实现通用数据访问层
- 4. 学会使用第三方 ORM 工具（Apache DbUtils）简化数据库操作
- 5. 理解参数化查询防止 SQL 注入的重要性

二、实验内容及步骤

2.1 JDBC 核心概念

概念说明

JDBC（Java Database Connectivity）是 Java 的标准数据库访问 API，包括四个核心组件：

组件	说明	用途
Driver	数据库驱动	连接到特定数据库的实现
Connection	数据库连接	建立与数据库的通信通道
Statement	SQL 执行器	执行 SQL 语句
ResultSet	结果集	存储和处理查询结果

工作流程

1. 注册驱动 → DriverManager.registerDriver()
↓
2. 获取连接 → Connection conn = DriverManager.getConnection()
↓
3. 创建语句 → Statement stmt = conn.createStatement()
↓
4. 执行 SQL → stmt.executeQuery() / executeUpdate()
↓
5. 处理结果 → ResultSet rs = stmt.getResultSet()
↓
6. 关闭资源 → rs.close() / stmt.close() / conn.close()

2.2 配置数据库连接池

Step 1: 创建 jdbc.properties 配置文件

```
# 数据库驱动类
driverClassName=org.h2.Driver

# 数据库连接地址（H2 内存数据库）
url=jdbc:h2:mem:schedule_db;MODE=MySQL;DB_CLOSE_DELAY=-1;DATABASE_TO_UPPER=false

# 数据库用户名和密码
username=sa
password=

# 连接池初始化连接数
initialSize=5

# 连接池最大连接数
maxActive=10
```

配置说明：

- driverClassName - 根据使用的数据库选择对应驱动
- url - 数据库连接地址，H2 使用内存数据库，MySQL 需指定主机和端口
- initialSize - 连接池启动时创建的初始连接数
- maxActive - 连接池允许创建的最大连接数

Step 2: 创建 JDBCUtil 工具类

```
package com.zpz.schedule.util;

import com.alibaba.druid.pool.DruidDataSourceFactory;
import javax.sql.DataSource;
import java.sql.Connection;
import java.sql.Statement;
import java.io.InputStream;
import java.sql.SQLException;
import java.util.Properties;

public class JDBCUtil {
    private static DataSource dataSource;

    // 静态初始化块：应用启动时执行一次
```

```

static {
    try {
        // 1. 加载 jdbc.properties 配置文件
        Properties props = new Properties();
        InputStream is =
JDBCUtil.class.getClassLoader().getResourceAsStream("jdbc.properties");
        props.load(is);

        // 2. 使用 Druid 创建连接池数据源
        dataSource = DruidDataSourceFactory.createDataSource(props);

        // 3. 自动建表 (如果表不存在)
        try (Connection conn = dataSource.getConnection();
            Statement stmt = conn.createStatement()) {

            // 建立 sys_user 表
            stmt.execute("CREATE TABLE IF NOT EXISTS sys_user (" +
                "uid INT PRIMARY KEY AUTO_INCREMENT, " +
                "username VARCHAR(50) NOT NULL UNIQUE, " +
                "user_pwd VARCHAR(64) NOT NULL" +
                ")");

            // 建立 sys_schedule 表
            stmt.execute("CREATE TABLE IF NOT EXISTS sys_schedule (" +
                "sid INT PRIMARY KEY AUTO_INCREMENT, " +
                "uid INT NOT NULL, " +
                "title VARCHAR(255), " +
                "completed INT DEFAULT 0, " +
                "CONSTRAINT fk_schedule_user FOREIGN KEY (uid)
REFERENCES sys_user(uid) ON DELETE CASCADE" +
                ")");

        }
    } catch (Exception e) {
        throw new RuntimeException("初始化数据库连接池失败", e);
    }
}

// 获取数据库连接
public static Connection getConnection() throws SQLException {
    return dataSource.getConnection();
}

// 获取数据源对象
public static DataSource getDataSource() {
    return dataSource;
}

```

```

// 关闭连接
public static void close(Connection conn) {
    if (conn != null) {
        try {
            conn.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
}
}

```

关键技术点：

- 使用 Druid 连接池管理数据库连接，提高性能
- 静态初始化块在应用启动时执行，确保连接池只初始化一次
- 自动 DDL：应用启动时自动创建数据库表
- try-with-resources 自动关闭资源

2.3 实现 DAO 设计模式

Step 1：创建 BaseDao 基类

```

package com.zpz.schedule.dao;

import org.apache.commons.dbutils.QueryRunner;
import org.apache.commons.dbutils.handlers.BeanHandler;
import org.apache.commons.dbutils.handlers.BeanListHandler;
import org.apache.commons.dbutils.handlers.ScalarHandler;
import com.zpz.schedule.util.JDBCUtil;

import java.sql.Connection;
import java.sql.SQLException;
import java.util.List;

public abstract class BaseDao {
    protected QueryRunner queryRunner = new QueryRunner();

    /**
     * 执行增删改操作
     */
    public int baseUpdate(String sql, Object... params) {
        Connection conn = JDBCUtil.getConnection();
        try {

```

```

        return queryRunner.update(conn, sql, params);
    } catch (SQLException e) {
        throw new RuntimeException(e);
    } finally {
        JDBCUtil.close(conn);
    }
}

/**
 * 查询单条记录
 */
public <T> T baseQueryOne(Class<T> clazz, String sql, Object... params)
{
    Connection conn = JDBCUtil.getConnection();
    try {
        return queryRunner.query(conn, sql, new BeanHandler<>(clazz),
params);
    } catch (SQLException e) {
        throw new RuntimeException(e);
    } finally {
        JDBCUtil.close(conn);
    }
}

/**
 * 查询多条记录
 */
public <T> List<T> baseQueryList(Class<T> clazz, String sql, Object...
params) {
    Connection conn = JDBCUtil.getConnection();
    try {
        return queryRunner.query(conn, sql, new BeanListHandler<>
(clazz), params);
    } catch (SQLException e) {
        throw new RuntimeException(e);
    } finally {
        JDBCUtil.close(conn);
    }
}

/**
 * 查询标量值 (如 COUNT、MAX 等)
 */
public Long baseQueryScalar(String sql, Object... params) {
    Connection conn = JDBCUtil.getConnection();
    try {

```

```

        return queryRunner.query(conn, sql, new ScalarHandler<>(),
params);
    } catch (SQLException e) {
        throw new RuntimeException(e);
    } finally {
        JDBCUtil.close(conn);
    }
}
}

```

设计优势：

- 通用的 CRUD 方法，避免重复代码
- 统一的异常处理和资源管理
- 使用 DbUtils 自动将 ResultSet 映射为对象

Step 2: 创建 DAO 接口

```

// SysUserDao 接口
public interface SysUserDao {
    int addSysUser(SysUser sysUser);
    SysUser findByUsername(String username);
}

// SysScheduleDao 接口
public interface SysScheduleDao {
    int addSchedule(SysSchedule schedule);
    List<SysSchedule> findByUid(Integer uid);
    int updateSchedule(SysSchedule schedule);
    int deleteSchedule(Integer sid);
}

```

Step 3: 实现 DAO 接口

SysUserDaoImpl:

```

package com.zpz.schedule.dao.impl;

import com.zpz.schedule.dao.BaseDao;
import com.zpz.schedule.dao.SysUserDao;
import com.zpz.schedule.pojo.SysUser;

public class SysUserDaoImpl extends BaseDao implements SysUserDao {

```

```

/**
 * 新增用户
 */
@Override
public int addSysUser(SysUser sysUser) {
    String sql = "INSERT INTO sys_user (username, user_pwd) VALUES (?, ?)";
    return baseUpdate(sql, sysUser.getUsername(), sysUser.getUserPwd());
}

/**
 * 按用户名查询用户
 * 注意: user_pwd 需要通过别名映射到 userPwd (驼峰命名)
 */
@Override
public SysUser findByUsername(String username) {
    String sql = "SELECT uid, username, user_pwd as userPwd FROM sys_user WHERE username = ?";
    return baseQueryOne(SysUser.class, sql, username);
}
}

```

SysScheduleDaoImpl:

```

package com.zpz.schedule.dao.impl;

import com.zpz.schedule.dao.BaseDao;
import com.zpz.schedule.dao.SysScheduleDao;
import com.zpz.schedule.pojo.SysSchedule;
import java.util.List;

public class SysScheduleDaoImpl extends BaseDao implements SysScheduleDao {

    /**
     * 新增日程
     */
    @Override
    public int addSchedule(SysSchedule schedule) {
        String sql = "INSERT INTO sys_schedule (uid, title, completed) VALUES (?, ?, ?)";
        return baseUpdate(sql, schedule.getUid(), schedule.getTitle(), schedule.getCompleted());
    }

    /**
     * 按用户ID查询日程列表
     */
}

```

```

    */
    @Override
    public List<SysSchedule> findByUid(Integer uid) {
        String sql = "SELECT sid, uid, title, completed FROM sys_schedule
WHERE uid = ?";
        return baseQueryList(SysSchedule.class, sql, uid);
    }

    /**
     * 更新日程状态
     */
    @Override
    public int updateSchedule(SysSchedule schedule) {
        String sql = "UPDATE sys_schedule SET title = ?, completed = ? WHERE
sid = ?";
        return baseUpdate(sql, schedule.getTitle(), schedule.getCompleted(),
schedule.getSid());
    }

    /**
     * 删除日程
     */
    @Override
    public int deleteSchedule(Integer sid) {
        String sql = "DELETE FROM sys_schedule WHERE sid = ?";
        return baseUpdate(sql, sid);
    }
}

```

关键技术点：

- 参数化查询：使用 ? 占位符和参数数组，防止 SQL 注入
- 列名别名：user_pwd as userPwd 实现数据库列名到 Java 属性名的映射
- 异常处理：子类继承 BaseDao 的异常处理机制

2.4 安全性实现 - 密码加密

```

package com.zpz.schedule.util;

import java.math.BigInteger;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;

public class MD5Util {
    /**

```

```

    * MD5 加密
    */
    public static String encrypt(String text) {
        try {
            MessageDigest md = MessageDigest.getInstance("MD5");
            byte[] messageDigest = md.digest(text.getBytes());
            BigInteger number = new BigInteger(1, messageDigest);
            String hashtext = number.toString(16);

            // 补零, 确保 32 位
            while (hashtext.length() < 32) {
                hashtext = "0" + hashtext;
            }
            return hashtext;
        } catch (NoSuchAlgorithmException e) {
            throw new RuntimeException("MD5 算法不支持", e);
        }
    }
}

```

三、实验结果

3.1 用户注册时的数据库操作

执行流程：

1. 用户提交注册表单
↓
2. SysUserServiceImpl.register() 验证
 - └ 检查用户名是否存在: SELECT uid FROM sys_user WHERE username = ?
 - └ 用户名不存在 → 继续
- ↓
3. 密码加密: MD5Util.encrypt()
↓
4. 插入数据库: INSERT INTO sys_user (username, user_pwd) VALUES (?, ?)
↓
5. 返回 true (成功) 或 false (失败)

数据库结果示例：

```

-- 执行注册后
INSERT INTO sys_user (username, user_pwd) VALUES ('zhangsan',
'5f4dcc3b5aa765d61d8327deb882cf99');
INSERT INTO sys_user (username, user_pwd) VALUES ('lisi',
'6512bd43d9caa6e02c990b0a82652dca');

```

-- 查询结果

```
SELECT uid, username, user_pwd FROM sys_user;
```

uid	username	user_pwd
1	zhangsan	5f4dcc3b5aa765d61d8327deb882cf99
2	lisi	6512bd43d9caa6e02c990b0a82652dca

3.2 用户登录时的数据库操作

执行流程：

1. 用户提交登录表单

↓

2. SysUserServiceImpl.login() 验证

└ 查询用户: SELECT uid, username, user_pwd as userPwd FROM sys_user WHERE username = ?

└ 用户不存在 → 返回 null

└ 用户存在 → 继续

↓

3. 密码校验

└ 加密输入密码: MD5Util.encrypt(inputPassword)

└ 比对数据库密码: dbPassword.equals(encryptedInput)

└ 相同 → 返回 SysUser 对象

└ 不同 → 返回 null

↓

4. 登录成功或失败

查询示例：

-- 用户名 'zhangsan' 的查询

```
SELECT uid, username, user_pwd as userPwd FROM sys_user WHERE username = 'zhangsan';
```

-- 结果

uid	username	userPwd
1	zhangsan	5f4dcc3b5aa765d61d8327deb882cf99

3.3 日程管理的 CRUD 操作

创建 (Create)：

```
-- 用户 1 添加日程
INSERT INTO sys_schedule (uid, title, completed) VALUES (1, '完成代码评审', 0);
```

-- 结果

sid	uid	title	completed
1	1	完成代码评审	0

读取 (Read):

-- 查询用户 1 的所有日程

```
SELECT sid, uid, title, completed FROM sys_schedule WHERE uid = 1;
```

-- 结果

sid	uid	title	completed
1	1	完成代码评审	0
2	1	审视测试用例	1

更新 (Update):

-- 标记日程 1 为已完成

```
UPDATE sys_schedule SET completed = 1 WHERE sid = 1;
```

删除 (Delete):

-- 删除日程 1

```
DELETE FROM sys_schedule WHERE sid = 1;
```

四、常见问题

无